DBMS ASSIGNMENT SERIES

Assignment 4

Relational Database vs NoSQL Database

Online Food Delivery System

SUBMITTED BY	PROGRAMME & INSTITUTION	SUBMISSION DATE
Vimal Kumar Rao	BCA (AI/ML) Alliance University	24 October 2026

A consistent relational design carried forward from Assignments 1 and 2.

IN THIS ASSIGNMENT

01	02	03	04
Compare models	Weigh trade-offs	Review cases	Choose the fit

1. Introduction

Relational and NoSQL databases solve overlapping but different data-management problems. A relational database organizes records into tables with declared relationships and constraints. NoSQL is a broad family that includes document, key-value, wide-column, and graph systems. NoSQL does not mean “no schema”; it usually means the model and query patterns differ from a traditional relational design.

2. Comparative Analysis

Dimension	Relational database (RDBMS)	NoSQL database	Practical implication
Data model	Tables, rows, columns, declared schema; relationships through PK/FK.	Documents, key-value pairs, wide columns, or graph structures; schema flexibility varies by product.	Use the model that matches how data is created and queried.
Scalability	Often scales vertically first; read replicas, partitioning, sharding, and distributed SQL can scale further.	Many products are designed for horizontal partitioning and elastic capacity across nodes.	Neither category is automatically “more scalable”; distribution model and workload determine the result.
Consistency	Relational engines commonly provide ACID transactions and declarative integrity rules. InnoDB supports ACID transactions [1].	Consistency varies by engine and operation. DynamoDB offers eventual consistency by default for many reads and supports strongly consistent reads in defined cases [2].	Payment/order updates often need clearly defined transactional guarantees.
Querying	SQL supports joins, grouping, subqueries, and ad-hoc analysis across related tables.	Queries often follow keys and access patterns; multi-entity joins may be limited or modeled through embedding/denormalization.	Choose based on common reads, write patterns, and reporting needs.
Performance	Can be efficient for joins, reporting, and transactions when schema, indexes, and queries are appropriate.	Can be very fast for well-defined high-volume key access and denormalized reads.	Benchmark representative workloads; do not assume a category wins universally.
Schema evolution	Schema changes are explicit and carefully managed.	Document models can allow varying fields and iterative evolution; validation can still be applied [3].	Flexible fields are useful for evolving product metadata, but validation remains important.
Best-fit examples	Orders, payments, inventory, billing, student records, booking systems.	Sessions, caches, telemetry, game progression, event streams, flexible product content.	Use relational integrity for transactional core data and NoSQL for suitable high-scale side workloads.

3. Real-World Examples

Relational example: the proposed food-delivery order system

The proposed application uses MySQL for customer identity, saved addresses, restaurant/menu references, order headers, order lines, and payment state. Keys and foreign keys prevent orphaned lines or payments; a transaction can coordinate critical state changes. Queries such as “orders by restaurant”, “customer spending”, and “paid order subtotal” require several related tables, so SQL joins and aggregate functions are natural fits.

NoSQL example: Ubisoft game-progression data on Amazon DynamoDB

AWS describes Ubisoft using DynamoDB for gameplay-progression data that supports challenges and leaderboards. The case study emphasizes adapting to variable traffic and a growing volume of gameplay data [4]. This is an example of a workload where high request volume and access patterns can justify a managed NoSQL database; it does not imply every part of the Ubisoft platform should be NoSQL.

NoSQL example: Prime Video services using DynamoDB

AWS reports that Prime Video migrated billions of rows from Oracle to DynamoDB for a globally replicated dataset associated with its video experience, improving latency and operational scalability [5]. This illustrates that technology choice can change as the workload, operations, and distribution requirements change.

4. Why Organizations Choose Different Technologies

- Data integrity needs: systems that must keep orders, payments, and inventory aligned often prioritize transactions, keys, and constraints.
- Access patterns: a service that reads one item by a known key millions of times per day may benefit from a purpose-built key-value/document data model.
- Volume and traffic shape: sudden spikes, worldwide access, and large event streams can favor distributed/managed NoSQL offerings, provided data access is designed around their partitioning model.
- Query complexity: ad-hoc joins and relational reporting favor SQL; fixed high-volume lookup patterns can favor denormalized NoSQL reads.
- Operational capability and cost: teams assess backup, monitoring, availability, licensing, managed services, skills, and migration costs rather than selecting by trend.
- Hybrid architecture: an organization can use SQL for transactional records, NoSQL for sessions/events/catalog metadata, and analytics storage for reporting. Each store should have a defined source of truth and a synchronization plan.

5. Recommendation for This Project

For the Online Food Delivery System coursework, MySQL is the recommended primary database. The schema has clear entity relationships and a transactional order workflow. A future production platform could add a NoSQL store for clickstream events, search suggestions, or short-lived sessions if measured load and access patterns justify it. Core orders and payment state should not be moved merely because NoSQL is popular; consistency and recovery requirements must drive the choice.

6. References

Ref.	Source	Used for in this report
[1]	MySQL 8.4 Reference Manual - InnoDB transactions and ACID properties	Transactions and relational integrity.
[2]	Amazon DynamoDB documentation - Read consistency	Strong and eventual consistency options.
[3]	MongoDB Manual - Data Modeling	Document design and modeling trade-offs.
[4]	AWS customer case study - Ubisoft and Amazon DynamoDB	Game-progression and leaderboard workloads.
[5]	AWS customer case study - Prime Video and Amazon DynamoDB	Large-scale, globally replicated workloads.
[6]	AWS - What is a NoSQL database?	Definitions and common NoSQL categories.

Citation key: Bracketed references in the report map to this table. Product capabilities and consistency options depend on the specific database service and configuration. Accessed 24 October 2026.